Prepare answers before the exercise class. In class you will present them.

### 3.1 Measuring clock differences of physical clocks

Measuring the difference between local clocks of two nodes is an important task in clock synchronization algorithms.

a) Explain a simple unidirectional and a simple bidirectional approach for this task. What is the content of the messages and how exactly is the difference calculated?
b) In each case, discuss if/what statements about accuracy (i.e., upper bound on the error) can be made!

### 3.2 CNV algorithm

Please download the file `exercise2-cnv.py`. It is a simple (partial) implementation of a simulation of the CNV algorithm. Try to understand the code (do not hesitate to ask if there are any doubts).

The code creates four instances of nodes (A, B, C, D) that use the CNV algorithm to synchronize their local clocks. By small code modifications, you can simulate different situations.

- The unmodified version has four clocks that start in perfect synchrony and have exactly the same speed. It should be no surprise that in this case, all nodes remain in perfect synchrony. You can even remove the line that invokes the CNV synchronization step (`n.sync(nodes)`), and still all will remain perfect.
- Change the node initialization such that one node will have an initial clock error and observe the behaviour of the algorithm.
- Change the node initialization such that one node will have a 5% frequency error and observe the behaviour.
- Change the code such that one node will use the "malicious" EVILCNV implementation (that generates random errors when reading the clock).

Note that in one of the cases, the observed behaviour will not match the expected behaviour (according to the lecture's description of the CNV algorithm).

a) Modify the algorithm such that it correctly implements the CNV algorithm, and repeat above experiment in which you observed a failure.
b) Modify the implementation of EVILCNV such that evil nodes cause maximum damage (maximise the `max diff` value printed during execution)

### 3.3 Lamport's CNV algorithm revisited

"Algorithm CNV. Each process reads the value of every process's clock and sets its own clock to the average of these values—except that if it reads a clock value differing from its own by more than $\delta$, then it replaces that value by its own clock's value when forming the average"[1]

(a) Assume you have 3 processes that execute the CNV algorithm, all of them correct. The parameter $\delta$ is 2 hours. In one synchronization round assume that:

- P1's clock reads 03:00, P2's clock reads 04:00, P3's clock reads 05:00

What will be the clock of P1, P2, and P3, respectively, after the synchronization?

(b) Assume now that P1 and P3 are the same as above, P2 is malicious. What behaviour of P2 has the worst impact on clock synchronization (i.e., maximizes the difference between P1's and P3's clock) in this example.

(c) Explain why the CNV algorithm fails to provide convergence in this example!.

### 3.4 Synchronisation of physical clocks

a) Which communication protocol (e.g., TCP, UDP, IP) would you use for implementing a clock synchronization algorithm, and why?

b) Would you implement physical clock synchronization in an application process, in the operating system, or in hardware? What are the respective advantages/disadvantages?

c) In the lecture, we discussed that a bidirectional measurement of time differences usually yields a higher accuracy than unidirectional approaches, because you can estimate (and compensate for) the message transmission delay based on measuring the round-trip message delay. Now lets assume that you want to further increase the accuracy by using multiple round-trip measurements.

Assume that the following four measurements $M_1 \ldots M_4$ have been made. Use and explain two criteria to select the best measurement that most likely minimizes the clock offset error. For the selected measurement, calculate the estimated clock offset E.

| base time B [hh:mm:ss] | $t_{C_1}$ [s] | $t_{S_1}$ [s] | $t_{S_2}$ [s] | $t_{C_2}$ [s] |
|---|---|---|---|---|
| M1: 12:05:00.000 | +0.0 | -2.450 | -2.350 | +0.200 |
| M2: 12:10:00.000 | +0.0 | -2.350 | -2.250 | +0.300 |
| M3: 12:15:00.000 | +0.0 | -2.450 | -2.350 | +0.300 |
| M4: 12:20:00.000 | +0.0 | -2.450 | -2.350 | +0.200 |

(Note: all time stamps are shown relative to the "base time", which is the start of the measurement. This means, for example, that for measurement M3, $t_{S_2}$ is 12:15:00.000 – 2.350s = 12:14:57.650.)

$t_{C_1}$: Client sends request

$t_{S_1}$: Server receives request

$t_{S_2}$: Server sends reply

$t_{C_2}$: Client receives reply

d) Daylight Savings Time (DST) cause "jumps" in local time (by +/- 1 hour once each year). How does the network time protocol account for this?t

[1] Leslie Lamport and P. M. Melliar-Smith. Synchronizing Clocks in the Presence of Faults. JACM 32(1), Jan 1985, https://lamport.azurewebsites.net/pubs/clocks.pdf